

Automating and Evaluating Cluster Analysis of X-Ray Diffraction

Olivia Frazier

Cornell University/CHEXS, oyf2@cornell.edu

Mentors: Suchismita Sarker, Jacob Ruff, Aaditya Panigrahi, Erik Andersen

Summary

X-ray Temperature-Clustering (*X-TEC*) is an unsupervised machine learning algorithm that analyzes large X-ray diffraction datasets to identify structural changes in a material across temperature (Venderley et al., 2022; KimGroup, 2022). By grouping reciprocal-space pixels that share similar intensity vs. temperature behavior, *X-TEC* reveals the physical transitions happening within a material without requiring prior crystal models. *XTEC_pipeline* combines the *X-TEC/XTEC-GPU* workflow in a single Jupyter notebook. One configuration cell controls data paths, GPU-accelerated hyperparameter sweeps, and figure generation. A downside of *X-TEC* is that it requires a user to choose the number of clusters, k . The standard approach has an expert analyzing diagnostic plots and picking k by eye, making it difficult for beamline scientists and visiting users to reach the same conclusion. *XTEC_pipeline* was built to reduce the need for expert review, so this work is about both building the pipeline and investigating whether choosing k can be automated. The result is negative, and it identifies a specific, previously unaddressed reason why: none of the metrics currently used account for the crystal's symmetry, which is what experts rely on when judging a clustering.

Statement of need

X-TEC provides an interpretable unsupervised framework for extracting order parameters from these datasets, but its standard execution is manual. Users must configure file paths, reciprocal-space slices, the algorithm variant (*XTEC-d* or *XTEC-s*), and clustering hyperparameters by hand, and repeat this setup for every script involved. *XTEC_pipeline* addresses this by collecting these settings into one place, a configuration cell at the top of the notebook, allowing users to update file paths and hyperparameters once instead of every time they appear. Analyzing a new dataset means editing one input path and material name, rather than updating file paths separately across multiple scripts.

The most significant remaining setting is the number of clusters, k . Determining a good k comes down to comparing the diagnostic plots and metrics from different candidate fits by eye, a process that is subjective and difficult to reproduce between beamline scientists and visiting users.

XTEC-GPU improves on this by providing two helper scripts, `sweep.py` and `judge.py`, that run GPU-accelerated Gaussian Mixture Model (GMM) fits across a range of k and scaling methods and score each fit with a weighted combination of quantitative metrics (KimGroup, 2025). In practice, when using these scripts, they are run as separate steps, each needing its own repeated configuration, and a user must cross-check the recommended k against diagnostic plots, because the scoring metrics and weights are not yet reliable enough to trust on their own. *XTEC_pipeline* was built to close these gaps, to combine data loading, sweeping, and diagnostic plotting into one workflow, and to test whether the judge's scoring can be improved to the point where it matches expert judgement.

State of the field

Two software packages currently support unsupervised temperature-dependent clustering of X-ray diffraction data. The original *X-TEC* package established the underlying method, GMM clustering of reciprocal-space pixels by their intensity vs. temperature trajectories, using standalone analysis scripts (KimGroup, 2022; Venderley et al., 2022). *XTEC-GPU* re-implements the core fitting routine on GPU and adds two utilities, `sweep.py` and `judge.py`, that respectively grid-search cluster counts and scaling methods and score the resulting fits (KimGroup, 2025). Both packages, however, treat hyperparameter search and evaluation as steps a user runs separately, each requiring its own command and its own repeated file paths and Q-region slices, and still rely on manual inspection of diagnostic plots to catch cases where `judge.py`'s scores disagree with what the data show.

There is no existing tool that unifies these stages into one reproducible workflow, and more importantly for this paper, no existing tool has tested whether `judge.py`'s scoring approach is actually trustworthy. *XTEC_pipeline* is not a reimplement of *X-TEC* or *XTEC-GPU*. It is a workflow layer built directly on top of them, contributing the missing integration between loading, sweeping, and diagnostics, and a test of the judge's metrics that neither upstream package's documentation addresses. Rewriting the underlying GMM fitting or GPU sweep code was not the goal and would duplicate existing, working implementations. The gap this project fills is streamlining and evaluation. That is why building a pipeline layer and diagnosing the judge's issues were the appropriate contribution rather than starting a new clustering package from scratch.

That test began with `judge.py` itself. It scores each candidate clustering with a hand-weighted combination of six components computed from the fit, including a normalized Bayesian Information Criterion (BIC) (Schwarz, 1978) term, a cluster-size penalty, a "jump" penalty, mean pairwise separation, mean trajectory smoothness, and a spatial coherence term. These weights were chosen by hand and overfit to the original dataset KimGroup used when developing `judge.py`. Two problems made this untrustworthy going into this project. First, the recommended k changed depending on whether the data was rescaled by mean or by z-score before fitting. Second, because the six component scores live on different scales, raw scores were not comparable across scaling methods in the first place. It was unclear whether the hand-tuned weights meant anything beyond the single dataset they were tuned on.

Software design

XTEC_pipeline runs a single Jupyter notebook, controlled by one configuration cell, where the user sets the data path, reciprocal-space slice bounds, scaling methods, and cluster range to sweep. From there the pipeline:

1. Discovers and stacks per-temperature reciprocal-space volumes into a single NeXus (.nxs) file, auto-detecting which temperature folders contain valid data and skipping the rest.
2. Slices and thresholds the stacked volume to the entered Q-region and removes low-intensity background (air scattering, detector noise) before model fitting.

3. Sweeps and scores GPU-accelerated GMM fits (via *XTEC-GPU*'s `sweep.py`) across the specified k range and rescaling methods, computing the judge's overall score for each fit directly inside the notebook.
4. Compiles diagnostics automatically after each sweep, including the log-intensity distribution used for background truncation, temperature trajectories for each cluster, and a Q-space map of cluster assignments, so an expert can visually confirm or override the recommended k without re-running anything.

The diagnostic plots of the pipeline are presented below and generated using the Ta_2NiSe_7 dataset. Figure 1 shows the log-intensity distribution, which separates the background noise from the signal (represented by all data points to the right of the red threshold dot). Figure 2 illustrates the temperature trajectories, tracking how clusters change as temperature increases. Figure 3 displays the Q-space map, a two-dimensional plot showing X-ray scattering intensity across different vectors of reciprocal space.

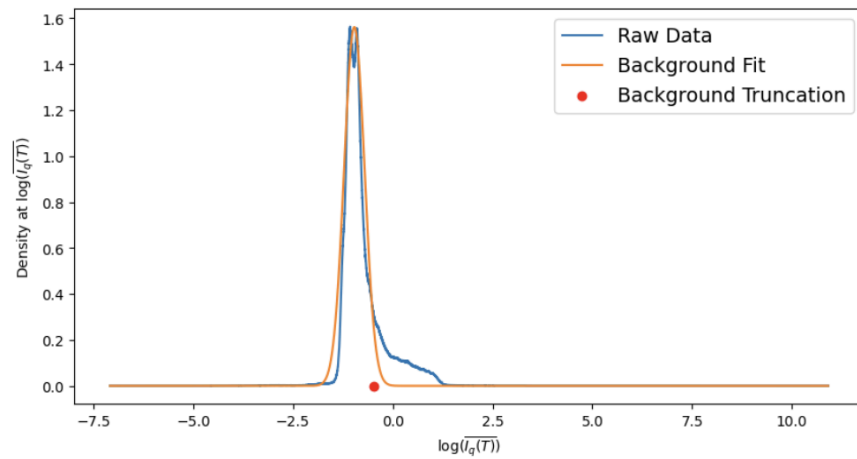


Figure 1. Log-intensity distribution of the Ta_2NiSe_7 dataset. The x-axis represents pixel intensity, and the y-axis represents frequency counts, separating background noise from signal to the right of the red threshold dot.

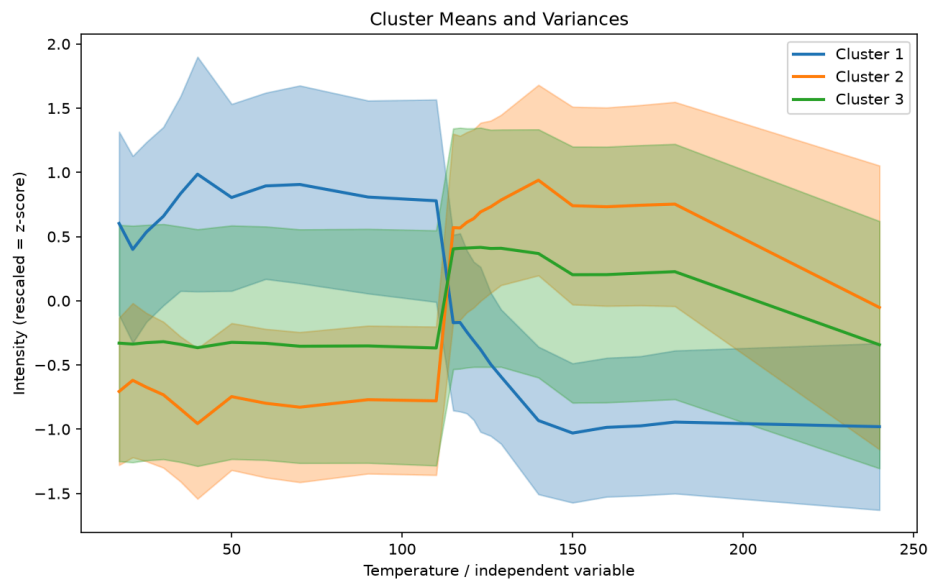


Figure 2. Temperature trajectories tracking change in clusters, which temperature on the x-axis and cluster assignment metrics on the y-axis.

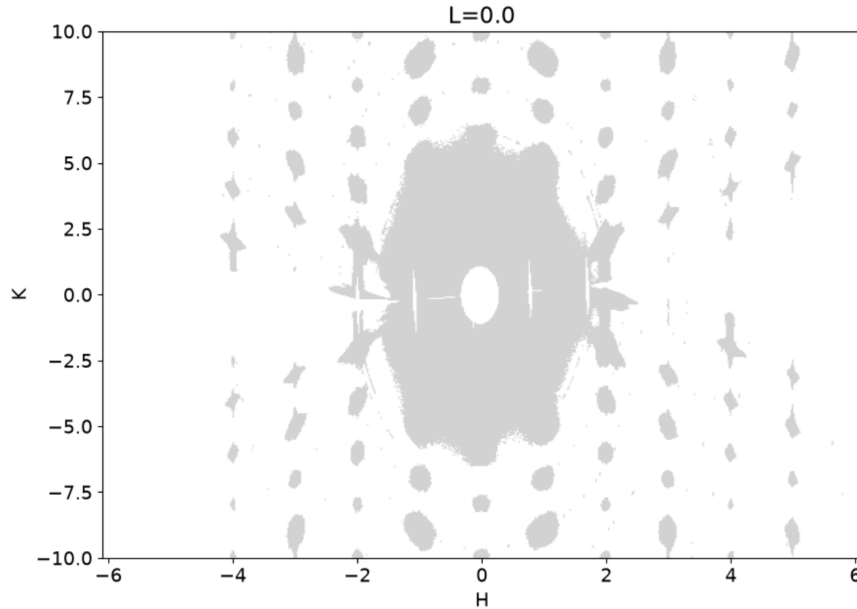


Figure 3. Q-space map plotted across reciprocal space vectors (Q_x and Q_y), where the color scale indicates scattering intensity to show crystal structures and cluster areas for the Ta_2NiSe_7 dataset.

To test whether the judge's weights could be made trustworthy, this project took three steps. First each of the six metric weights was swept independently across a range of values while holding the others fixed, to measure how much each one actually influenced the recommended k . Second, five additional metrics were implemented and evaluated, including a BIC-elbow detector, cluster amplitude, signal-to-noise ratio, redundancy between clusters, and Q-space compactness. Third, a Bradley-Terry model (Bradley & Terry, 1952) was trained on synthetic datasets with known cluster counts, in an attempt to learn metric weights directly from data rather than by hand.

The weight sweeps showed that out of the six original weights tested, only one actually shifted the recommended k . Two of the six had no measurable effect on the outcome at all. This alone was a useful finding, since most of the judge's metrics were not doing anything. However, the larger result was negative. None of the original hand-tuned score, the five candidate metrics, or the weights learned by the Bradley-Terry model consistently agreed with expert judgement across the test datasets. No single metric, and no learned combination of metrics, reliably reproduced the k that an expert would choose.

By looking at the current and candidate metrics, we can see that they only evaluate intensity vs. temperature trajectories and Q-space occupancy, while ignoring crystal symmetry. Since experts rely on this to validate cluster boundaries, automated scoring will continue to fail until it also accounts for symmetry.

Research impact statement

XTEC_pipeline is designed for ongoing operational use at the Q-mapping for Quantum Materials (QM2) beamline, where it will be used by beamline scientists and visiting users. By replacing several separate script calls with one configuration-driven notebook, it removes a real limitation at the QM2 beamline.

This reproducible workflow makes it easier to analyze complex datasets, like what it was tested on ($\text{Ta}_2\text{NiSeS}_7$), without requiring code changes between materials.

Just as important is what the judge investigation found. The recurring pattern in the disagreements was informative. The automated metrics tended to favor clusterings that were statistically well-separated but crystallographically meaningless, splitting a single physical order parameter into two clusters because it produced a better BIC, or merging two Q-space regions that are visually and thermodynamically distinct. Every metric tested in this study is computed from only intensity vs. temperature trajectories and Q-space occupancy. None of them accounts for the material's space group. A likely reason is that an expert evaluating a clustering is also weighing whether cluster boundaries in Q-space respect symmetry operations of the crystal. Q-points related by symmetry should behave alike, and a clustering that splits them apart is treated as a modeling artifact, not a real physical distinction, regardless of what the BIC score says. None of the metrics tested here satisfy this requirement. This means that automated cluster scoring cannot happen until it also accounts for crystal symmetry.

Automated cluster scoring cannot be expected to match expert judgement until it accounts for symmetry, which no previously tested metrics had. Instead of testing more statistical metrics and reweighting there is a possibility that a trustworthy judge will require encoding crystal symmetry directly into cluster scoring.

Code Availability

The code developed for this work has been made publicly available at the GitHub link:

<https://github.com/oliviafrazier3-design/CHESS-Summer-School-QM2-XTEC>

AI usage disclosure

Generative AI tools (Claude) were used to assist in developing some code, debugging, and creating synthetic datasets to test on. All AI-generated code and analysis were reviewed, tested on real datasets, and verified by the author. Other than spelling, grammar checking, and help shortening some prose, this paper was written without AI assistance.

Acknowledgements

I would like to thank Suchismita Sarker, Jacob Ruff, Aaditya Panigrahi, and Erik Andersen for their guidance, feedback, and support during the development of this software. This material is based on work supported by the AI Research Institutes program supported by the NSF and Intel Corporation under NSF award DMR-2433348. This work is based on research conducted at the Center for High-Energy X-ray Sciences (CHEXS), which is supported by the National Science Foundation (BIO, ENG and MPS Directorates) under award DMR-2342336.

References

- Bradley, R. A., & Terry, M. E. (1952). Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4), 324–345.
- KimGroup. (2022). XTEC [Software]. GitHub. <https://github.com/KimGroup/XTEC>
- KimGroup. (2025). XTEC-GPU [Software]. GitHub. <https://github.com/KimGroup/XTEC-GPU>
- Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2), 461-464.
- Venderley, J., Mallayya, K., Matty, M., Krogstad, M., Ruff, J., Pleiss, G., Kishore, V., Mandrus, D., Phelan, D., Poudel, L., Wilson, A. G., Weinberger, K., Upreti, P., Norman, M., Rosenkranz, S., Osborn, R., & Kim, E.-A. (2022). Harnessing interpretable and unsupervised machine learning to address big data from modern X-ray diffraction. *Proceedings of the National Academy of Sciences*, 119(24), e2109665119. <https://doi.org/10.1073/pnas.2109665119>